



Tecnicatura Universitaria en Programación Base de Datos II

Equipo # 27

Integrantes:

**Sena, Sofia Guadalupe- 31873
Scapolla, Bruno - 33057
Bernal, Nicolas - 33053
Brandan, Franco- 31782**

Explicación del Sistema

El sistema será una aplicación web para la gestión, interacción y reproducción de contenido audiovisual.

La base de datos almacenará información sobre Usuario, Contenidos, Actores, Géneros, Productora, y tipos de contenidos.

Para optimizar el diseño, se manejará una tabla llamada 'Contenido', la cual permitirá diferenciar entre películas, series o documentales mediante un tipo asociado.

Las series, también contarán con dos tablas adicionales Temporadas y Episodios, que permite organizar los episodios según su temporada.

Los usuarios podrán registrarse, buscar contenido según género, productora, año o actores principales, marcar contenido como favorito, calificarlo, dejar comentarios y crear playlists personalizadas. Además, podrán seguir a distintas productoras y visualizar su historial de reproducciones.

La funcionalidad de visualizaciones se aplicará solo a películas, permitiendo registrar los datos como la cantidad de vistas.

Funcionalidades principales

1. Módulo de Gestión de Usuarios y Suscripciones

❖ Usuarios

- **Función:** Almacena la información personal y credenciales de acceso de los clientes de la plataforma (nombre, DNI, email, país).
- **Se relaciona con:** [SuscripcionDelUsuario](#), [Visualizacion](#), [Favoritos](#), [Calificacion](#), [Comentario](#), [Playlist](#), [SeguirProductora](#)
- **¿Por qué?:** Es la entidad central de la experiencia. Todo el historial de reproducciones, pagos e interacciones en la plataforma debe estar vinculado unívocamente a un usuario específico.

❖ Suscripción

- **Función:** Define los diferentes planes o membresías disponibles en la plataforma (ej. Básico, Premium), incluyendo su precio y descripción.
- **Se relaciona con:** [SuscripcionDelUsuario](#).
- **¿Por qué?:** Permite escalar el modelo de negocio. Si el día de mañana se crea un plan "Familiar", solo se agrega un registro aquí sin modificar la estructura de los usuarios.

❖ **SuscripcionDelUsuario**

- **Función:** Actúa como tabla intermedia para registrar qué usuario tiene qué suscripción, incluyendo las fechas de inicio, vencimiento y si está activa.
- **Se relaciona con:** **Usuarios** y **Suscripcion**.
- **¿Por qué?:** Resuelve la relación de muchos a muchos (N:M) a lo largo del tiempo. Un usuario puede cambiar de plan Básico a Premium a lo largo de su vida, y esta tabla permite mantener un historial de facturación y acceso sin perder datos pasados.

2. Módulo de Catálogo de Contenidos (El Eje Central)

❖ **Contenido**

- **Función:** Es la tabla principal del catálogo. Centraliza toda la información general de una obra audiovisual (título, duración, fecha de lanzamiento).
- **Se relaciona con:** **TipoContenido**, **Genero**, **Productora**, **Temporada**, **ContenidoActor** y todas las tablas de interacción de los usuarios.
- **¿Por qué?:** Al unificar películas, series y documentales en una sola tabla, se simplifican enormemente las búsquedas, los algoritmos de recomendación y el registro de reproducciones.

❖ **TipoContenido y Genero**

- **Función:** Son tablas de catálogo (o paramétricas) que definen la naturaleza de la obra (ej. Película o Serie) y su género (ej. Acción, Drama).
- **Se relacionan con:** **Contenido**.
- **¿Por qué?:** Normalizan la base de datos. Evitan errores de tipeo (ej. escribir "Acción" sin tilde y "Acción" con tilde) y permite crear filtros de búsqueda eficientes.

❖ **Productora**

- **Función:** Almacena la información de los dueños o creadores del contenido (ej. Netflix, Marvel Studios).
- **Se relaciona con:** **Contenido** y **SeguirProductora**.
- **¿Por qué?:** Permite que la plataforma ofrezca portales a las productoras para ver sus estadísticas, además de agrupar contenido por creador para los usuarios.

❖ **Temporada y Episodio**

- **Función:** Manejan la jerarquía específica de las series. **Temporada** agrupa episodios de un año/ciclo, y **Episodio** detalla el capítulo específico.
- **Se relacionan con:** **Temporada** se vincula a **Contenido**, y **Episodio** se vincula a **Temporada**.
- **¿Por qué?:** Es un diseño altamente estructurado. Una película no tendrá temporadas, por lo que esas tablas quedan vacías para ese tipo de contenido, pero para una serie, permite una navegación ordenada sin mezclar los metadatos de la serie general con la duración de un capítulo individual.

❖ **Actores y ContenidoActor**

- **Función:** **Actores** guarda la ficha del profesional, y **ContenidoActor** es la tabla intermedia que une al actor con el proyecto en el que trabajó.
- **Se relacionan con:** **ContenidoActor** une a **Actores** con **Contenido**.
- **¿Por qué?:** Resuelve la relación de muchos a muchos. Una película tiene muchos actores, y un actor trabaja en muchas películas a lo largo de su carrera.

3. Módulo de Interacción y Experiencia del Usuario

❖ **Visualización**

- **Función:** Registra de manera exclusiva el historial de reproducciones de contenidos unitarios de una sola pieza (películas o documentales), guardando qué usuario lo vio y en qué fecha.
- **Se relaciona con:** Las tablas **Usuarios** y **Contenido**.
- **¿Por qué?:** Permite almacenar el historial de reproducciones de los usuarios y generar estadísticas sobre los contenidos más vistos.

❖ **Favoritos, Calificación y Comentario**

- **Función:** Capturan el *feedback* directo del cliente. Si le dio "Me gusta", si le puso puntaje (1 al 10) o si escribió una reseña.
- **Se relacionan con:** **Usuarios** y **Contenido**.
- **¿Por qué?:** Estas tablas intermedias conectan al espectador con la obra. Separarlas permite que un usuario pueda calificar algo sin comentarlo, o agregarlo a favoritos sin haberlo visto aún.

❖ **Playlist y PlaylistContenido**

- **Función:** **Playlist** guarda el encabezado de una lista creada por el usuario (ej. "Tarde de Acción"), y **PlaylistContenido** es el detalle de los videos dentro de esa lista.

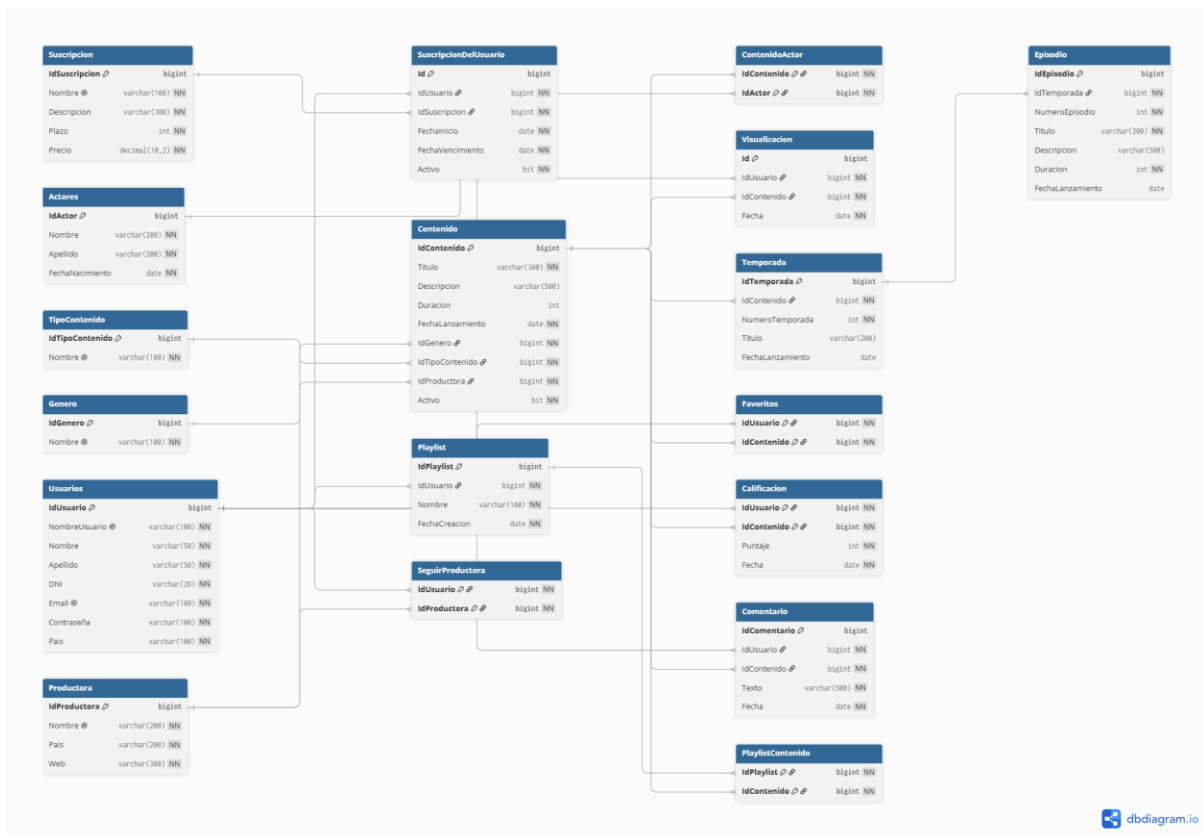
- **Se relacionan con:** **Playlist** pertenece a **Usuarios**.
PlaylistContenido une **Playlist** con **Contenido**.
- **¿Por qué?:** Resuelve la relación N:M. Un contenido puede estar en muchas playlists diferentes, y una playlist está conformada por múltiples contenidos.

❖ **SeguirProductora**

- **Función:** Registra cuando un usuario decide suscribirse a las novedades de un estudio específico.
- **Se relaciona con:** **Usuarios** y **Productora**
- **¿Por qué?:** Permite personalizar la experiencia del usuario mostrando contenido relacionado con las productoras que sigue.

El sistema está construido con una base de datos relacional que almacenará información sobre Usuarios, Películas, series, documentales, genero, productora, accesos y tipo de contenido, lo cual permitirá personalizar la experiencia y mejorar la gestión general de la plataforma.

Diagrama de Entidad Relación



Objetos de Base de Datos clave en el Sistema

Vista 1

Se utiliza la vista **VW_Contenido**, que muestra los contenidos activos disponibles en la plataforma. Esta vista está destinada a facilitar la consulta del contenido por parte de los administradores y usuarios.

Contiene la siguiente información: título del contenido, descripción, duración, tipo de contenido, género y productora.

```
CREATE VIEW VW_Contenido As
SELECT
    C.Titulo,
    C.Descripcion,
    C.Duracion,
    T.Nombre AS 'Tipo Contenido',
    G.Nombre AS 'Genero',
    P.Nombre AS 'Productora'
FROM Contenido C
INNER JOIN TipoContenido T ON T.IdTipoContenido = C.IdTipoContenido
INNER JOIN Genero G ON C.IdGenero = G.IdGenero
INNER JOIN Productora P ON C.IdProductora = P.IdProductora
WHERE C.Activo = 1;
```

Vista 2

Se utiliza la vista **VW_ComentariosContenido**, para mostrar los comentarios más recientes de los usuarios sobre cualquier tipo de contenido (películas, series).

Nos permite ver título del contenido, nombre de usuario, comentario y fecha del comentario.

```
CREATE VIEW VW_ComentariosContenido AS
SELECT
    C.Titulo,
    U.NombreUsuario,
    CO.Texto,
    CO.Fecha
FROM Comentario CO
INNER JOIN Usuarios U ON CO.IdUsuario = U.IdUsuario
INNER JOIN Contenido C ON CO.IdContenido = C.IdContenido
```

Vista 3

Se utiliza la vista **VW_UsuariosActivos**, que muestra los usuarios con suscripciones activas. Esta vista está destinada a ser consultada por los administradores. Contiene la siguiente información: id del usuario, nombreUsuario, apellido, nombre, email y fecha de vencimiento.

```
CREATE VIEW VW_UsuariosActivos AS
SELECT
    U.IdUsuario AS 'IdUsuario',
    U.NombreUsuario,
    U.Apellido,
    U.Nombre,
    U.Email,
    US.FechaVencimiento
FROM Usuarios U
INNER JOIN SuscripcionDelUsuario US ON US.IdUsuario = U.IdUsuario
WHERE US.Activo = 1;
```

Vista 4

Se utiliza la vista **VW_HistorialVisualizacion**, para consultar el historial de reproducciones realizadas por los usuarios. Esta vista está destinada a facilitar la visualización de la actividad de los usuarios y contiene la siguiente información: nombre del usuario, título y fecha de visualización

```
CREATE VIEW VW_HistorialVisualizacion
AS
SELECT
    U.NombreUsuario,
    C.Titulo,
    V.Fecha
FROM Visualizacion V
INNER JOIN Usuarios U ON V.IdUsuario = U.IdUsuario
INNER JOIN Contenido C ON V.IdContenido = C.IdContenido
```

Procedimiento Almacenado 1

Se utiliza el procedimiento almacenado **SP_ContratarSuscripcion**, para registrar una nueva suscripción a un usuario. El mismo recibe como parámetro IdUsuario y el IdSuscripcion, obtiene automáticamente el plazo correspondiente, calcula la fecha de inicio y la fecha de vencimiento automáticamente.

```

CREATE PROCEDURE SP_ContratarSuscripcion
    @IdUsuario BIGINT,
    @IdSuscripcion BIGINT
AS
BEGIN
    BEGIN TRY
        -- Verificar que el usuario ya exista
        BEGIN TRANSACTION
        IF EXISTS(
            SELECT 1
            FROM SuscripcionDelUsuario
            WHERE IdUsuario = @IdUsuario AND
            Activo = 1 AND
            FechaVencimiento >= CAST(getdate() AS date)
        )
        BEGIN
            RAISERROR('ERROR: ESTE USUARIO YA TIENE UNA SUSCRIPCION ACTIVA',16,1)
            ROLLBACK TRANSACTION;
            RETURN
        END
        DECLARE @Plazo INT;
        SELECT @Plazo = Plazo
        FROM Suscripcion
        WHERE IdSuscripcion = @IdSuscripcion;
        INSERT INTO SuscripcionDelUsuario
        (
            IdUsuario,
            IdSuscripcion,
            FechaInicio,
            FechaVencimiento
        )
        VALUES
        (
            @IdUsuario,
            @IdSuscripcion,
            GETDATE(),
            DATEADD(DAY, @Plazo, GETDATE())
        );
        COMMIT TRANSACTION;
        PRINT 'SUSCRIPCION CONTRATADA CORRECTAMENTE';
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        PRINT ERROR_MESSAGE();
    END CATCH;
END;

```


Procedimiento Almacenado 2

Se utiliza el procedimiento almacenado **SP_RegistrarVisualizacion**, para registrar las visualizaciones realizadas por los usuarios. El mismo recibe como parámetro el IdUsuario y el IdContenido, almacenando la reproducción en el historial de visualizaciones. Está destinado a ser utilizado por la aplicación cada vez que un usuario reproduce una película o serie.

```
Create Procedure SP_RegistrarVisualizacion
    @IdUsuario bigint,
    @IdContenido bigint
As
Begin
    If not exists (
        Select 1
        From Visualizacion
        Where IdUsuario @IdUsuario=
            AND IdContenido @IdContenido=
            AND Fecha = CAST(GETDATE() AS DATE)
    )
    Begin
        Insert into Visualizacion(IdUsuario, IdContenido)
        Values(@IdUsuario, @IdContenido);

        Print 'Visualización registrada correctamente.';
    End
    Else
    Begin
        Print 'La visualización ya fue registrada para este usuario en el día de hoy.';
    End
End;
```

Procedimiento Almacenado 3

Se utiliza el procedimiento almacenado **SP_ContenidoPorPaisProductora** para consultar los contenidos activos producidos en un país determinado. El procedimiento recibe como parámetro el nombre del país y devuelve información del contenido. Está destinado a facilitar búsquedas y reportes por ubicación geográfica de las productoras.

```

CREATE PROCEDURE SP_ContenidoPorPaisProductora
    @Pais VARCHAR(100)
AS
BEGIN
    SELECT
        C.Titulo,
        C.Descripcion,
        P.Nombre AS Productora,
        P.Pais,
        G.Nombre AS Genero,
        T.Nombre AS TipoContenido
    FROM Contenido C
    INNER JOIN Productora P ON C.IdProductora = P.IdProductora
    INNER JOIN Genero G ON C.IdGenero = G.IdGenero
    INNER JOIN TipoContenido T ON C.IdTipoContenido = T.IdTipoContenido
    WHERE P.Pais = @Pais
        AND C.Activo = 1
END

```

Procedimiento Almacenado 4

Se utiliza el procedimiento almacenado **SP_AgregarFavorito** para incorporar un contenido a la lista de favoritos de un usuario. El procedimiento recibe como parámetros el IDUsuario y el IDContenido, verificando previamente que la asociación no exista para evitar duplicados

```

CREATE PROCEDURE SP_AgregarFavorito
    @IdUsuario BIGINT,
    @IdContenido BIGINT
AS
BEGIN
    -- Validamos que no exista ya en favoritos
    IF NOT EXISTS (
        SELECT 1
        FROM Favoritos
        WHERE IdUsuario @IdUsuario =
            AND IdContenido @IdContenido=
    )
    BEGIN
        INSERT INTO Favoritos (IdUsuario, IdContenido)
        VALUES (@IdUsuario, @IdContenido)
        PRINT 'CONTENIDO AGREGADO A FAVORITOS';
    END
    ELSE
    BEGIN
        PRINT 'EL CONTENIDO YA SE ENCUENTRA EN LA LISTA DE FAVORITOS';
    END
END

```

Procedimiento Almacenado 5

Se utiliza el procedimiento almacenado **SP_RegistrarNuevoUsuario** para dar de alta a un nuevo cliente en la plataforma. El mismo recibe como parámetros la información personal y las credenciales de acceso (nombre de usuario, nombre, apellido, DNI, email, país y contraseña), valida automáticamente que no existan registros previos con el mismo DNI, email o usuario para evitar duplicados, y ejecuta la inserción de los datos.

```

CREATE PROCEDURE SP_RegistrarNuevoUsuario
    @NombreUsuario VARCHAR(100),
    @Nombre VARCHAR(50),
    @Apellido VARCHAR(100),
    @DNI VARCHAR(20),
    @Email VARCHAR(100),
    @Pais VARCHAR(50),
    @Contraseña VARCHAR(100)
AS
BEGIN
    BEGIN TRANSACTION;
    -- Verificar si el usuario ya existe por DNI o Email
    IF EXISTS (SELECT 1 FROM Usuarios WHERE dni @DNI =OR email @Email)=
    BEGIN
        -- Si existe
        RAISERROR('Error: Ya existe un usuario registrado con ese DNI o Email.', 16, 1);
    END
    -- Inserción en la tabla de Usuarios
    INSERT INTO Usuarios (NombreUsuario, nombre, Apellido, dni, email, pais, Contraseña)
    VALUES (@NombreUsuario, @Nombre, @Apellido, @DNI, @Email, @Pais, @Contraseña);
    -- Si todo salió bien
    COMMIT TRANSACTION;
    PRINT 'Usuario registrado con éxito en el sistema.';
END;

```

Trigger 1

Se utiliza el trigger **TR_BajaLogica_Contenido** para realizar una baja lógica de los contenidos. Cuando se intenta eliminar una película o serie, el trigger impide la eliminación física del registro y actualiza su estado a inactivo. De esta manera se conserva la información y se evita la pérdida de datos.

```

CREATE TRIGGER TR_BajaLogica_Contenido On Contenido
INSTEAD OF DELETE
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION

        DECLARE @IdContenido BIGINT
        SELECT @IdContenido = IdContenido From deleted

        UPDATE Contenido SET Activo = 0 WHERE IdContenido = @IdContenido

        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION
    END CATCH
END

```

Trigger 2

Se utiliza el trigger **TR_Favoritos_ValidarContenido** para validar que el contenido agregado a favoritos se encuentre disponible. Si una película o serie está marcada como inactiva, el trigger cancela la operación, evitando que los usuarios agreguen contenidos no disponibles a su lista de favoritos.

```

CREATE TRIGGER TR_Favoritos_ValidarContenido
ON Favoritos
AFTER INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN Contenido C ON C.IdContenido = i.IdContenido
        WHERE C.Activo = 0
    )
    BEGIN
        ROLLBACK TRANSACTION;
    END
END

```

Trigger 3

Se utiliza el trigger **TR_ValidarTemporadaContenido** para mantener la integridad de las reglas. Cuando se intenta insertar o actualizar un registro en la tabla de temporadas, el trigger evalúa la categoría del contenido asociado. Si detecta que se trata de una película, el trigger impide la acción y revierte la transacción.

```
CREATE TRIGGER TR_ValidarTemporadaContenido ON Temporada
AFTER INSERT, UPDATE
AS
BEGIN
    -- Evita interferencias con el conteo de filas afectadas
    SET NOCOUNT ON;

    -- Verifica si alguna de las filas insertadas o modificadas pertenece a una Película
    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN Contenido c ON i.IdContenido = c.IdContenido
        INNER JOIN TipoContenido tc ON c.IdTipoContenido = tc.IdTipoContenido
        WHERE tc.nombre LIKE '%Películas%'
    )
    BEGIN
        -- Si encuentra una coincidencia, cancela la transacción y muestra el error
        RAISERROR('No está permitido asignar temporadas ni episodios a contenidos catalogados como Película.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

Función 1

Se utiliza la función **FN_PromedioCalificacion** para calcular el promedio de las calificaciones otorgadas por los usuarios a un contenido. La función recibe como parámetro el identificador del contenido y devuelve el puntaje promedio correspondiente, permitiendo conocer la valoración general de películas y series dentro de la plataforma.

```
CREATE FUNCTION FN_PromedioCalificacion
(
    @IdContenido BIGINT
)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @Promedio DECIMAL(5,2);

    SELECT @Promedio = AVG(CAST(Puntaje AS DECIMAL(5,2)))
    FROM Calificacion
    WHERE IdContenido = @IdContenido;

    RETURN ISNULL(@Promedio, 0);
END;
```

Links a los recursos

Script de creación de bases de datos

https://github.com/SofiaGuada/BD_027_TP

Video demo del sistema

https://drive.google.com/file/d/1Q0UgL92sts_TetxEaOYlOxZLWOYAqgly/view?usp=sharing